

NumPy — Nền tảng tính toán cho AI

Thư viện cho Trí tuệ nhân tạo — Bài 1

Trình bày: Võ Luyện

Nội dung buổi học

- 1 Vì sao AI cần NumPy?
- 2 Bắt đầu với NumPy
- 3 Các cách tạo mảng
- 4 Thuộc tính và kiểu dữ liệu
- 5 Truy cập phần tử: indexing và slicing
- 6 View và Copy — cái bẫy kinh điển
- 7 Vector hóa — tính toán không cần vòng lặp
- 8 Broadcasting
- 9 Hàm thống kê và khái niệm axis
- 10 Biến đổi hình dạng và ghép mảng
- 11 Đại số tuyến tính cho AI
- 12 NumPy trong các bài toán AI
- 13 Lỗi thường gặp và mẹo hay
- 14 Thực hành

Dữ liệu trong AI là những con số — rất nhiều con số

- Một tấm ảnh 224×224 màu $\rightarrow 224 \times 224 \times 3 = 150\,528$ con số
- Một bảng điểm của 1000 học sinh với 10 môn $\rightarrow 10\,000$ con số
- Một mô hình mạng nơ-ron nhỏ $\rightarrow$ hàng triệu tham số cần cập nhật liên tục

Dữ liệu trong AI là những con số — rất nhiều con số

- Một tấm ảnh 224×224 màu $\rightarrow 224 \times 224 \times 3 = 150\,528$ con số
- Một bảng điểm của 1000 học sinh với 10 môn $\rightarrow 10\,000$ con số
- Một mô hình mạng nơ-ron nhỏ $\rightarrow$ hàng triệu tham số cần cập nhật liên tục

Câu hỏi

Nếu dùng `list` của Python thuần và vòng lặp `for` để xử lý từng con số, chuyện gì xảy ra?

Dữ liệu trong AI là những con số — rất nhiều con số

- Một tấm ảnh 224×224 màu $\rightarrow 224 \times 224 \times 3 = 150\,528$ con số
- Một bảng điểm của 1000 học sinh với 10 môn $\rightarrow 10\,000$ con số
- Một mô hình mạng nơ-ron nhỏ $\rightarrow$ hàng triệu tham số cần cập nhật liên tục

Câu hỏi

Nếu dùng `list` của Python thuần và vòng lặp `for` để xử lý từng con số, chuyện gì xảy ra?

Trả lời

Chương trình chạy **rất chậm** — huấn luyện một mô hình có thể mất vài ngày thay vì vài phút.

NumPy là gì?

- **NumPy** (Numerical Python) — thư viện tính toán số học nền tảng của Python
- Ra đời năm 2006, hiện là "trái tim" của toàn bộ hệ sinh thái khoa học dữ liệu và AI
- Cung cấp kiểu dữ liệu **ndarray** — mảng nhiều chiều, lưu số hiệu quả
- Phần lõi được viết bằng **C** và Fortran → nhanh hơn Python thuần hàng chục đến hàng trăm lần

NumPy là gì?

- **NumPy** (Numerical Python) — thư viện tính toán số học nền tảng của Python
- Ra đời năm 2006, hiện là "trái tim" của toàn bộ hệ sinh thái khoa học dữ liệu và AI
- Cung cấp kiểu dữ liệu **ndarray** — mảng nhiều chiều, lưu số hiệu quả
- Phần lõi được viết bằng **C** và Fortran → nhanh hơn Python thuần hàng chục đến hàng trăm lần

Ai đang dùng NumPy?

Pandas, Scikit-learn, Matplotlib, OpenCV, PyTorch, TensorFlow — tất cả đều xây dựng trên NumPy hoặc dùng chung cách tổ chức dữ liệu của NumPy.

Vị trí của NumPy trong hệ sinh thái AI

PyTorch / TensorFlow — Deep Learning

Scikit-learn — Machine Learning cổ điển

Pandas
dữ liệu dạng bảng

Matplotlib
vẽ biểu đồ

NumPy — mảng số nhiều chiều ndarray

↑
Càng xuống dưới
càng nền tảng

- Học NumPy tốt → học các thư viện phía trên nhanh hơn rất nhiều

So sánh tốc độ: list vs ndarray

```
import numpy as np, time

n = 1_000_000
list_a = list(range(n))
arr_a = np.arange(n)

t0 = time.time()
tong1 = sum(x * x for x in list_a)    # Python thuan
t1 = time.time()
tong2 = np.sum(arr_a * arr_a)         # NumPy
t2 = time.time()

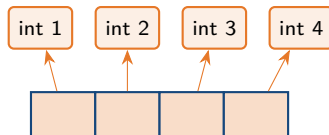
print("Python list:", t1 - t0, "giây")
print("NumPy      :", t2 - t1, "giây")
```

Kết quả tham khảo (tùy máy)

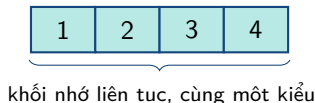
Python list $\approx$ 0.025–0.09 giây; NumPy $\approx$ 0.001–0.002 giây → **nhANH hơn 20–50 lần**

Vì sao NumPy nhanh hơn?

Python list



NumPy array



Mỗi ô chỉ là **con trỏ** tới một đối tượng Python nằm rải rác trong bộ nhớ. Mỗi phép cộng phải: tìm đối tượng → kiểm tra kiểu → tính → tạo đối tượng mới.

Các số nằm **liền nhau**, cùng kiểu → vòng lặp chạy trong C, tận dụng cache CPU và lệnh xử lý song song (SIMD).

NumPy array khác Python list ở điểm nào?

	Python list	NumPy ndarray
Kiểu phần tử	Trộn lẫn tùy ý	Đồng nhất (int, float, ...)
Kích thước	Thay đổi được	Cố định khi tạo
Bộ nhớ	Nhiều, rải rác	Ít, liên tục
Tốc độ tính toán	Chậm (vòng lặp)	Nhanh (chạy trong C)
Phép toán số học	$[1, 2] + [3, 4] \rightarrow$ nối	$a + b \rightarrow$ cộng từng phần tử
Nhiều chiều	Phải lồng list	Hỗ trợ sẵn
Hàm toán học	Phải tự viết	Có sẵn hàng trăm hàm

Ghi nhớ

List dùng để chứa **mọi thứ**. NumPy array dùng để **tính toán trên số**.

Cài đặt và import

Cài đặt (chỉ làm một lần, trong Terminal / Command Prompt):

```
pip install numpy
```

Với Google Colab hoặc Anaconda: NumPy đã được cài sẵn. **Import** — quy ước chung của cả thế giới:

```
import numpy as np  
  
print(np.__version__)  # kiểm tra phiên bản
```

Lưu ý

Luôn viết `import numpy as np`. Mọi tài liệu, mọi khóa học, mọi đoạn code trên mạng đều dùng bí danh `np`.

Mảng đầu tiên của bạn

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
print(a)           # [1 2 3 4 5]
print(type(a))     # <class 'numpy.ndarray'>

# Phep toan tac dong len TOAN BO mang
print(a + 10)      # [11 12 13 14 15]
print(a * 2)       # [ 2  4  6  8 10]
print(a ** 2)      # [ 1  4  9 16 25]
```

- Không cần vòng lặp `for` — đây gọi là **vector hóa** (vectorization)
- So sánh: với list, `[1,2,3] * 2` cho `[1,2,3,1,2,3]` — hoàn toàn khác!

Mảng nhiều chiều — khái niệm cốt lõi

3	1	4	1
---	---	---	---

1 chiều (1D)

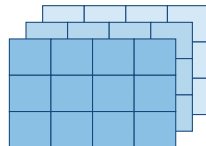
vector — shape (4,)

vd: điểm 4 môn học

2 chiều (2D)

ma trận — shape (3,4)

vd: bảng điểm 3 HS $\times$ 4 môn



3 chiều (3D)

tensor — shape (3,4,3)

vd: ảnh màu cao $\times$ rộng $\times$ RGB

- Mọi dữ liệu trong AI — ảnh, âm thanh, văn bản, bảng số — đều được đưa về mảng nhiều chiều

Tạo mảng 2 chiều

```
diem = np.array([[8.0, 7.5, 9.0],      # hoc sinh 1
                 [6.5, 8.0, 7.0],      # hoc sinh 2
                 [9.0, 9.5, 8.5]])      # hoc sinh 3

print(diem)
print(diem.shape)    # (3, 3)  -> 3 hang, 3 cot
print(diem.ndim)     # 2       -> so chieu
```

- Mảng 2D được tạo từ **list của các list**
- Mỗi list con là một **hàng** (row) và phải có **cùng độ dài**
- Quy ước đọc shape: (số hàng, số cột)

Tạo mảng từ giá trị có sẵn

```
np.zeros(5)           # [0. 0. 0. 0. 0.]
np.zeros((2, 3))      # ma tran 2x3 toan so 0
np.ones((2, 3))       # ma tran 2x3 toan so 1
np.full((2, 3), 7)    # ma tran 2x3 toan so 7
np.eye(3)             # ma tran don vi 3x3 (duong cheo = 1)
np.empty((2, 2))      # chua khoi tao - gia tri rac, rat nhanh
```

- Chú ý **dấu ngoặc kép**: `np.zeros((2, 3))` — hình dạng là một **tuple**
- `np.zeros` rất hay dùng để khởi tạo trọng số hoặc bộ đếm trong AI

Lỗi thường gặp

`np.zeros(2, 3)` → báo lỗi. Tham số thứ hai là **dtype**, không phải số cột!

Tạo dãy số: arange và linspace

```
np.arange(5)           # [0 1 2 3 4]
np.arange(2, 10)       # [2 3 4 5 6 7 8 9]
np.arange(0, 10, 2)    # [0 2 4 6 8]          buoc nhay = 2
np.arange(0, 1, 0.25)  # [0.  0.25 0.5  0.75]

np.linspace(0, 1, 5)   # [0.  0.25 0.5  0.75 1.  ]
np.linspace(0, 10, 3)  # [ 0.  5. 10.]
```

	Bạn chỉ định	Điểm cuối
<code>np.arange(a, b, buoc)</code>	bước nhảy	không lấy b
<code>np.linspace(a, b, n)</code>	số phần tử	có lấy b

- Vẽ đồ thị hàm số → dùng `linspace`; tạo chỉ số đếm → dùng `arange`

Tạo mảng ngẫu nhiên

```
rng = np.random.default_rng(42)    # 42 là "hat giống" (seed)

rng.random(5)                       # 5 số thực ngẫu nhiên trong [0, 1)
rng.random((2, 3))                  # ma trận 2x3 số thực ngẫu nhiên
rng.integers(1, 7, 10)               # 10 số nguyên 1..6 (giao xúc xắc)
rng.normal(0, 1, 5)                 # 5 số phân phối chuẩn (mean 0, std 1)
```

Vì sao cần seed?

Đặt seed → lần chạy nào cũng cho **cùng một kết quả** → thí nghiệm AI có thể **lặp lại được** (reproducible). Đây là nguyên tắc bắt buộc khi làm nghiên cứu.

- Cách cũ (vẫn gặp nhiều trong tài liệu): `np.random.seed(42)` rồi `np.random.rand(5)`

Các thuộc tính quan trọng của mảng

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])

print(a.shape)      # (2, 3)    hình dạng: 2 hàng, 3 cột
print(a.ndim)       # 2        số chiều
print(a.size)        # 6        tổng số phần tử
print(a.dtype)       # int64    kiểu dữ liệu của phần tử
print(a.itemsize)    # 8        số byte mỗi phần tử
print(a.nbytes)      # 48       tổng số byte = size * itemsize
```

Thói quen tốt số 1 khi làm AI

Gặp lỗi hay nghi ngờ → `print(x.shape)` ngay lập tức. Hơn 80% lỗi khi lập trình AI là do **sai hình dạng mảng**.

Kiểu dữ liệu (dtype)

```
np.array([1, 2, 3]).dtype           # int64 (Windows cũ: int32)
np.array([1.0, 2.0]).dtype          # float64
np.array([1, 2], dtype=np.float32) # ép kiểu ngay khi tạo

a = np.array([1.7, 2.3, 3.9])
b = a.astype(int)                   # [1 2 3] - CAT phân thập phân
c = a.astype(np.float32)
```

Kiểu	Bộ nhớ	Dùng khi
uint8	1 byte	Ảnh (giá trị 0–255)
int32/int64	4/8 byte	Nhãn, chỉ số, số đếm
float32	4 byte	Chuẩn của Deep Learning
float64	8 byte	Mặc định của NumPy, độ chính xác cao
bool	1 byte	Mặt nạ lọc dữ liệu

Cẩn thận với kiểu số nguyên

```
a = np.array([250, 251, 252], dtype=np.uint8)
print(a + 10)          # [4 5 6] ??? -- TRAN SƠ (overflow)!
# uint8 chỉ chứa được 0..255, 260 quay vòng về 4

b = np.array([1, 2, 3])
b[0] = 3.9
print(b)               # [3 2 3] -- bị cắt thành số nguyên
```

Quy tắc an toàn khi làm AI

Trước khi tính toán trên ảnh, luôn đổi sang số thực: `img = img.astype(np.float32) / 255.0`

Indexing và slicing mảng 1 chiều

```
a = np.array([10, 20, 30, 40, 50, 60])
```

```
a[0]          # 10    phan tu dau tien  
a[-1]         # 60    phan tu cuoi cung  
a[1:4]        # [20 30 40] tu chi so 1 den 3  
a[:3]         # [10 20 30] 3 phan tu dau  
a[3:]         # [40 50 60] tu chi so 3 den het  
a[::2]        # [10 30 50] cach 1 lay 1  
a[::-1]       # [60 50 40 30 20 10] dao nguoc
```

- Cú pháp giống hệt list của Python: `a[start : stop : step]`
— bắt đầu, kết thúc, bước nhảy
- Luôn nhớ: chỉ số bắt đầu từ **0**, và **không** lấy phần tử ở vị trí `stop`

Indexing mảng 2 chiều

```
a = np.array([[ 1,  2,  3,  4],
               [ 5,  6,  7,  8],
               [ 9, 10, 11, 12]])

a[1, 2]      # 7      hang 1, cot 2
a[1][2]      # 7      cach viet cua list - cham, TRANH DUNG
a[0]         # [1 2 3 4]    toan bo hang 0
a[:, 1]      # [2 6 10]    toan bo cot 1
a[0:2, 1:3]  # [[2 3], [6 7]] mang con 2x2
a[-1, -1]   # 12      phan tu goc duoi phai
```

Cách đọc

Dấu phẩy ngăn cách các chiều: `a[i, j]` — hàng `i`, cột `j`. Dấu hai chấm : nghĩa là "lấy tất cả".

Minh họa slicing 2 chiều

1	2	3	4
5	6	7	8
9	10	11	12

`a[0]`

1	2	3	4
5	6	7	8
9	10	11	12

`a[:, 1]`

1	2	3	4
5	6	7	8
9	10	11	12

`a[0:2, 1:3]`

Lọc dữ liệu bằng điều kiện (boolean masking)

```
diem = np.array([8.5, 4.0, 7.0, 9.5, 5.5, 3.0])

mask = diem >= 5
print(mask)           # [True False True True True False]
print(diem[mask])     # [8.5 7.  9.5 5.5]   chỉ lấy diem >= 5
print(diem[diem >= 5]) # viết gọn, rất phổ biến

print(np.sum(diem >= 5)) # 4   - đếm số học sinh đạt
print(np.mean(diem >= 5)) # 0.666... - tỉ lệ đạt

# Kết hợp nhiều điều kiện: dùng & (và), | (hoặc), ~ (phủ định)
print(diem[(diem >= 5) & (diem < 9)]) # [8.5 7.  5.5]
```

Chú ý

Dùng **&**, **|** chứ **không** dùng **and**, **or**. Và **bắt buộc** có dấu ngoặc quanh mỗi điều kiện.

Fancy indexing và np.where

Lấy nhiều phần tử theo danh sách chỉ số:

```
a = np.array([10, 20, 30, 40, 50])
a[[0, 2, 4]]           # [10 30 50]
a[[4, 4, 0]]           # [50 50 10] - lặp lại được, thu tu tùy ý
```

np.where — "nếu ...thì ...ngược lại":

```
diem = np.array([8.5, 4.0, 7.0, 3.0])

np.where(diem >= 5, 1, 0)      # [1 0 1 0] - gan nhan Dat/Truot
np.where(diem >= 5, diem, 0)   # [8.5 0. 7. 0.] - truot -> 0
np.where(diem >= 5)            # (array([0, 2]),) - vi tri
```

- np.where chính là cách viết if-else cho cả mảng, không cần vòng lặp

Slicing tạo ra "view", không phải bản sao

```
a = np.array([1, 2, 3, 4, 5])
b = a[1:4]          # b là VIEW - nhìn vào cùng vùng nhớ với a
b[0] = 99

print(b)            # [99  3  4]
print(a)            # [ 1 99  3  4  5]  <-- a CÙNG BI ĐỔI!
```

Slicing tạo ra "view", không phải bản sao

```
a = np.array([1, 2, 3, 4, 5])
b = a[1:4]          # b là VIEW - nhìn vào cùng vùng nhớ với a
b[0] = 99

print(b)            # [99  3  4]
print(a)            # [ 1 99  3  4  5]  <-- a CÙNG BI ĐOI!
```

Muốn có bản sao độc lập → dùng `.copy()`:

```
c = a[1:4].copy()
c[0] = 0
print(a)            # [ 1 99  3  4  5]  <-- a không đổi
```

Vì sao NumPy làm vậy?

Để **tiết kiệm bộ nhớ** — cắt một mảng 1GB mà phải sao chép thì rất tốn kém.

Khi nào là view, khi nào là copy?

Thao tác	Kết quả
<code>b = a[1:4]</code>	View — dùng chung bộ nhớ
<code>b = a[a > 5]</code>	Copy (boolean masking)
<code>b = a[[0, 2]]</code>	Copy (fancy indexing)
<code>b = a.reshape(2, 3)</code>	View (nếu có thể)
<code>b = a.T</code>	View
<code>b = a.copy()</code>	Copy
<code>b = a + 0</code>	Copy (mảng mới từ phép toán)

```
b = a[1:4]
print(b.base is a)    # True  -> b là view của a
```

Quy tắc vàng cho người mới

Nếu bạn định **sửa** dữ liệu và **không** muốn ảnh hưởng mảng gốc → luôn gọi `.copy()`.

Phép toán trên từng phần tử (element-wise)

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

a + b      # [11 22 33 44]
b - a      # [ 9 18 27 36]
a * b      # [ 10 40 90 160]  nhan TUNG PHAN TU
b / a      # [10. 10. 10. 10.]
a ** 2     # [ 1  4  9 16]
b % 3      # [1 2 0 1]

a > 2      # [False False  True  True]
```

- Hai mảng phải có **cùng hình dạng** (hoặc tuân theo quy tắc broadcasting ở phần sau)
- Kết quả luôn là một mảng **mới**, cùng hình dạng

Các hàm toán học có sẵn (universal functions)

```
a = np.array([1, 4, 9, 16])

np.sqrt(a)          # [1.  2.  3.  4.]      can bac hai
np.exp(a)           # e^a                ham mu
np.log(a)           # ln(a)              logarit tu nhien
np.abs(-a)          # gia tri tuyet doi
np.round(np.array([1.234, 5.678]), 1)    # [1.2 5.7]

x = np.linspace(0, np.pi, 5)
np.sin(x)           # ham luong giac
np.maximum(a, 5)    # [ 5  5  9 16]      so sanh tung phan tu voi 5
```

Vì sao quan trọng với AI?

Các hàm kích hoạt (activation) của mạng nơ-ron — sigmoid, tanh, ReLU, softmax — đều được viết chỉ bằng vài hàm này.

Vector hóa: viết lại vòng lặp cho gọn và nhanh

Cách viết của người mới (chậm):

```
ket_qua = []  
for i in range(len(diem)):  
    ket_qua.append(diem[i] * 2 + 1)  
ket_qua = np.array(ket_qua)
```

Cách viết NumPy (nhanh hơn hàng chục lần):

```
ket_qua = diem * 2 + 1
```


Vector hóa: viết lại vòng lặp cho gọn và nhanh

Cách viết của người mới (chậm):

```
ket_qua = []  
for i in range(len(diem)):  
    ket_qua.append(diem[i] * 2 + 1)  
ket_qua = np.array(ket_qua)
```

Cách viết NumPy (nhanh hơn hàng chục lần):

```
ket_qua = diem * 2 + 1
```

Nguyên tắc số 1 khi lập trình AI với NumPy

Thấy vòng lặp `for` chạy trên từng phần tử của mảng → hãy nghĩ cách viết lại bằng phép toán trên cả mảng.

Broadcasting — khi hai mảng khác hình dạng

```
a = np.array([1, 2, 3])
print(a + 10)           # [11 12 13] - so 10 duoc "phat" ra ca mang

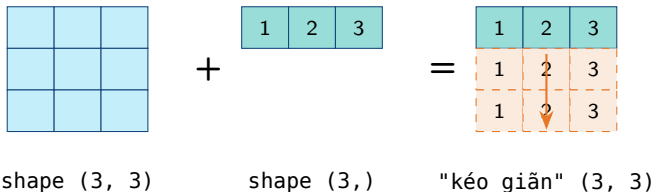
diem = np.array([[8.0, 7.0, 9.0],      # 3 hoc sinh
                 [6.0, 8.5, 7.5],      # x 3 mon
                 [9.0, 9.0, 8.0]])
he_so = np.array([2, 1, 1])           # Toan he so 2, Ly 1, Hoa 1

print(diem * he_so)                  # moi HANG deu duoc nhan voi he_so
```

Broadcasting là gì?

NumPy tự động "kéo giãn" mảng nhỏ hơn cho khớp hình dạng mảng lớn — **không tốn thêm bộ nhớ**, không cần vòng lặp.

Minh họa broadcasting



- Hàng [1 2 3] được lặp lại cho **mọi hàng** của ma trận
- Việc "kéo giãn" chỉ là **khái niệm** — NumPy không thực sự tạo bản sao trong bộ nhớ

Quy tắc broadcasting

So sánh hình dạng **từ phải sang trái**. Hai chiều tương thích khi:

- 1 Chúng **bằng nhau**, hoặc
- 2 Một trong hai **bằng 1**, hoặc
- 3 Một mảng **không có** chiều đó (được coi như bằng 1)

Hình dạng A	Hình dạng B	Kết quả
(3, 4)	(4,)	(3, 4) — OK
(3, 4)	(3, 1)	(3, 4) — OK
(3, 1)	(1, 4)	(3, 4) — OK, giãn cả hai
(3, 4)	(3,)	Lỗi — 4 và 3 không khớp

```
ValueError: operands could not be broadcast together  
with shapes (3,4) (3,)
```

Ứng dụng broadcasting: chuẩn hóa dữ liệu

Chuẩn hóa (normalization) là bước **bắt buộc** trước khi huấn luyện mô hình:

```
# X: 100 mau du lieu, moi mau co 4 dac trung (features)
X = rng.normal(50, 10, size=(100, 4))

mean = X.mean(axis=0)    # shape (4,) - trung binh TUNG COT
std  = X.std(axis=0)     # shape (4,) - do lech chuan tung cot

X_chuan = (X - mean) / std    # broadcasting: (100,4) voi (4,)

print(X_chuan.mean(axis=0))  # ~ [0. 0. 0. 0.]
print(X_chuan.std(axis=0))   # ~ [1. 1. 1. 1.]
```

- Chỉ **một dòng** thay cho hai vòng lặp lồng nhau và 400 phép tính thủ công
- Đây gọi là chuẩn hóa **z-score** — công thức $z = (x - \mu) / \sigma$

Các hàm tổng hợp

```
a = np.array([3, 1, 4, 1, 5, 9, 2, 6])

a.sum()          # 31      tong
a.mean()         # 3.875  trung binh
a.min()          # 1      gia tri nho nhat
a.max()          # 9      gia tri lon nhat
a.std()          # do lech chuan
a.argmin()       # 1      VI TRI cua gia tri nho nhat
a.argmax()       # 5      VI TRI cua gia tri lon nhat
np.median(a)     # 3.5    trung vi
a.cumsum()       # [ 3  4  8  9 14 23 25 31] tong tích luy
```

argmax — hàm quan trọng bậc nhất trong AI

Mô hình phân loại trả về xác suất cho từng lớp; **argmax** cho biết **lớp nào có xác suất cao nhất** → đó là dự đoán của mô hình.

axis — chiều nào bị "gộp lại"?

	1	2	3	4
	5	6	7	8
	9	10	11	12

Lệnh	Ý nghĩa	Kết quả
<code>a.sum()</code>	tổng tất cả	78
<code>a.sum(axis=0)</code>	gộp các hàng → tổng mỗi cột	[15 18 21 24]
<code>a.sum(axis=1)</code>	gộp các cột → tổng mỗi hàng	[10 26 42]

Mẹo nhớ

`axis=k` nghĩa là "chiều thứ k **biến mất**". Mảng $(3, 4)$ với `axis=0` → còn lại $(4,)$.

Ví dụ thực tế với axis

```
# 4 hoc sinh x 3 mon (Toan, Ly, Hoa)
diem = np.array([[8.0, 7.0, 9.0],
                 [6.0, 8.5, 7.5],
                 [9.0, 9.0, 8.0],
                 [5.0, 6.0, 6.5]])

diem.mean(axis=1)      # DTB của từng HỌC SINH -> 4 giá trị
diem.mean(axis=0)      # DTB của từng MÔN -> 3 giá trị
diem.max(axis=0)       # điểm cao nhất mỗi môn
diem.argmax(axis=0)    # học sinh giỏi nhất mỗi môn (chỉ số)

# Giữ nguyên số chiều để tiện broadcasting
tb = diem.mean(axis=1, keepdims=True) # shape (4,1) ko (4,)
print(diem - tb)             # do lệch số với DTB của chính mình
```


reshape — thay đổi hình dạng

```
a = np.arange(12)           # [0 1 2 ... 11]   shape (12,)
b = a.reshape(3, 4)         # 3 hàng, 4 cột
c = a.reshape(4, 3)         # 4 hàng, 3 cột
d = a.reshape(2, 2, 3)      # mang 3 chiều
e = a.reshape(3, -1)        # -1 = "tự tính giúp tôi" -> (3, 4)

print(b.ravel())             # [0 1 ... 11]   tra về 1 chiều (view)
print(b.flatten())           # giống ravel nhưng tra về BAN SÀO
```

- Số phần tử phải **không đổi**: $3 \times 4 = 12$ (được), $5 \times 3 = 15$ (lỗi)
- -1 cực kỳ hay dùng: `X.reshape(-1, 784)` — "gộp ảnh thành vector 784 chiều, số ảnh tự tính"

Lỗi thường gặp

`ValueError: cannot reshape array of size 12 into shape (5,3)`

Chuyển vị và thêm chiều

```
a = np.array([[1, 2, 3],  
              [4, 5, 6]])          # shape (2, 3)  
  
a.T                                # chuyển vị -> shape (3, 2)  
a.transpose()                     # giống a.T  
  
v = np.array([1, 2, 3])           # shape (3,) - không hàng, không cột  
v[:, np.newaxis]                  # shape (3, 1) - vector CỘT  
v[np.newaxis, :]                  # shape (1, 3) - vector HÀNG  
v.reshape(-1, 1)                  # cách viết khác cho vector cột
```

Điểm dễ nhầm

Mảng shape (3,) khác shape (3,1) và khác shape (1,3). Rất nhiều lỗi broadcasting đến từ chỗ này — nhớ `print(x.shape)`!

Ghép và tách mảng

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

np.concatenate([a, b], axis=0)    # ghép theo HÀNG -> (4, 2)
np.concatenate([a, b], axis=1)    # ghép theo CỘT -> (2, 4)
np.vstack([a, b])                 # vertical = axis 0
np.hstack([a, b])                 # horizontal = axis 1
np.stack([a, b])                  # tạo chiều MỚI -> (2, 2, 2)

x = np.arange(10)
np.split(x, 2)                    # tách làm 2 phần bằng nhau
np.split(x, [3, 7])               # tách tại vị trí 3 và 7
```

- `np.stack` thêm một chiều mới — dùng khi gom nhiều ảnh thành một "batch"

Nhân ma trận — phép toán trung tâm của Deep Learning

```
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [7, 8]])

A * B      # [[ 5 12], [21 32]]    nhan TUNG PHAN TU
A @ B      # [[19 22], [43 50]]    NHAN MA TRAN (dung nhut)
np.dot(A, B)      # giong A @ B
np.matmul(A, B)   # giong A @ B
```

Đừng nhầm!

* là nhân từng phần tử. @ là nhân ma trận theo toán học. Trong AI, @ mới là phép dùng để tính đầu ra của một lớp mạng nơ-ron.

- Điều kiện: $(m, n) @ (n, p) \rightarrow (m, p)$ — số cột của A phải bằng số hàng của B

Các phép đại số tuyến tính khác

```
u = np.array([1, 2, 3])
v = np.array([4, 5, 6])

np.dot(u, v)           # 32    tích vô hướng (dot product)
np.linalg.norm(u)      # 3.74  độ dài vector (chuẩn Euclid)
np.linalg.norm(u - v)  # khoảng cách giữa hai điểm

A = np.array([[2, 1], [1, 3]])
np.linalg.inv(A)        # ma trận nghịch đảo
np.linalg.det(A)        # định thức
np.trace(A)             # vết (tổng đường chéo)
np.linalg.solve(A, np.array([5, 10])) # giải hệ  $Ax = b$ 
```

- `np.linalg.norm` dùng để đo **độ giống nhau** giữa hai dữ liệu — nền tảng của thuật toán k-NN và tìm kiếm ngữ nghĩa

Ứng dụng: một lớp mạng nơ-ron

Công thức của một lớp fully-connected: $y = XW + b$

```
rng = np.random.default_rng(0)

X = rng.normal(size=(5, 3))      # 5 mau, moi mau 3 dac trung
W = rng.normal(size=(3, 4))      # trong so: 3 vao -> 4 no-ron
b = np.zeros(4)                  # do lech (bias) cho 4 no-ron

Y = X @ W + b                    # broadcasting cho b
print(Y.shape)                   # (5, 4)

def relu(x):
    return np.maximum(0, x)

A = relu(Y)                      # ham kích hoạt
```

- Đây chính xác là những gì PyTorch/TensorFlow làm bên trong — chỉ khác là chúng chạy trên GPU

Ảnh số chính là một mảng NumPy

```
# Ảnh mau: (cao, rộng, 3 kênh RGB), kiểu uint8 giá trị 0..255
img = rng.integers(0, 256, size=(128, 128, 3), dtype=np.uint8)

img.shape          # (128, 128, 3)
img[0, 0]          # [R G B] của điểm ảnh góc trên trái

img_gray = img.mean(axis=2)          # ảnh xám: trung bình 3 kênh
img_flip = img[:, ::-1, :]          # lật ảnh trái-phải
img_crop = img[32:96, 32:96]         # cắt vùng giữa
img_norm = img.astype(np.float32) / 255.0 # đưa về [0, 1]

do_sang = img.mean()                # độ sáng trung bình
kênh_do = img[:, :, 0]              # tách riêng kênh đỏ
```

- Lật ảnh, cắt ảnh, xoay ảnh — kỹ thuật **tăng cường dữ liệu** (data augmentation) — đều chỉ là slicing

Các hàm kích hoạt viết bằng NumPy

```
def sigmoid(x):  
    return 1 / (1 + np.exp(-x))  
  
def relu(x):  
    return np.maximum(0, x)  
  
def tanh(x):  
    return np.tanh(x)  
  
def softmax(x):  
    e = np.exp(x - np.max(x))    # tru max de tranh tran so  
    return e / np.sum(e)  
  
diem_so = np.array([2.0, 1.0, 0.1])  
print(softmax(diem_so))          # [0.659 0.242 0.099] - tong = 1
```

- softmax biến các con số bất kỳ thành **xác suất** — bước cuối của mọi mô hình phân loại

Hàm mất mát (loss function)

```
y_that = np.array([3.0, 5.0, 2.5, 7.0]) # gia tri dung
y_du_doan = np.array([2.8, 5.2, 3.0, 6.5]) # mo hinh du doan

# Sai so binh phuong trung binh (MSE)
mse = np.mean((y_that - y_du_doan) ** 2)
print(mse) # 0.145

# Sai so tuyet doi trung binh (MAE)
mae = np.mean(np.abs(y_that - y_du_doan))

# Do chinh xac cho bai toan phan loai
nhan_that = np.array([0, 1, 1, 0, 1])
nhan_du_doan = np.array([0, 1, 0, 0, 1])
acc = np.mean(nhan_that == nhan_du_doan) # 0.8 -> 80%
```

- Huấn luyện mô hình = tìm tham số làm cho **giá trị loss nhỏ nhất**

Ứng dụng: thuật toán k láng giềng gần nhất (k-NN)

```
X = rng.normal(size=(100, 3))      # 100 mẫu đa biến nhân
y = rng.integers(0, 2, 100)        # nhãn 0 hoặc 1
mau_moi = rng.normal(size=3)       # mẫu cần dự đoán

# Khoảng cách đến TAT CA các mẫu - không dùng vòng lặp!
kc = np.linalg.norm(X - mau_moi, axis=1) # shape (100,)

k = 5
lang_gieng = np.argsort(kc)[:k]     # chỉ số của 5 mẫu gần nhất
nhân = y[lang_gieng]
du_doan = np.bincount(nhân).argmax() # nhãn phổ biến nhất
print("Dự đoán:", du_doan)
```

- Một thuật toán Machine Learning hoàn chỉnh trong **5 dòng** nhờ vector hóa

Ứng dụng: chia dữ liệu train / test

```
X = rng.normal(size=(100, 4))
y = rng.integers(0, 2, 100)

chi_so = np.arange(len(X))
rng.shuffle(chi_so)           # xáo trộn chỉ số

n_train = int(0.8 * len(X))   # 80% để huấn luyện
train_idx = chi_so[:n_train]
test_idx  = chi_so[n_train:]

X_train, y_train = X[train_idx], y[train_idx] # fancy indexing
X_test,  y_test  = X[test_idx],  y[test_idx]

print(X_train.shape, X_test.shape) # (80, 4) (20, 4)
```

- Xáo trộn **chỉ số** chứ không xáo trộn dữ liệu → **X** và **y** luôn khớp nhau

Ứng dụng: hồi quy tuyến tính với gradient descent

Tìm w, b sao cho $y \approx wx + b$:

```
x = np.linspace(0, 10, 50)
y = 3 * x + 5 + rng.normal(0, 1, 50)    # du lieu that: w=3, b=5

w, b = 0.0, 0.0
lr = 0.01                                # toc do hoc

for buoc in range(1000):
    y_pred = w * x + b
    loi = y_pred - y
    dw = 2 * np.mean(lois * x)            # dao ham theo w
    db = 2 * np.mean(lois)                # dao ham theo b
    w -= lr * dw
    b -= lr * db

print(f"w = {w:.2f}, b = {b:.2f}")        # w ~ 3.0, b ~ 5.0
```

Toàn bộ Deep Learning được xây trên đúng ý tưởng này, chỉ ở quy mô lớn hơn rất nhiều.

Bốn lỗi kinh điển của người mới

1. Sai hình dạng khi nhân ma trận

ValueError: matmul: Input operand 1 has a mismatch **in** core dim

→ `print(A.shape, B.shape)`: cột của A phải bằng hàng của B

2. Quên rằng slicing là view

```
b = a[:3]          # sửa b -> a cũng đổi. Dùng a[:3].copy()
```

3. Dùng `and` / `or` với mảng

ValueError: The truth value of an array **with** more than one element **is** ambiguous

→ dùng `&`, `|` kèm dấu ngoặc; hoặc `np.any()`, `np.all()`

4. So sánh số thực bằng `==`

```
0.1 + 0.2 == 0.3          # False !  
np.isclose(0.1 + 0.2, 0.3) # True  <- cách dùng
```

Mẹo giúp code nhanh và sạch

- Luôn in shape khi debug: `print(x.shape, x.dtype)`
- Tránh vòng lặp trên phần tử — tìm cách viết bằng phép toán mảng
- Đừng nối mảng trong vòng lặp — mỗi lần `np.append` là một lần sao chép toàn bộ:

```
# CHAM
kq = np.array([])
for i in range(10000):
    kq = np.append(kq, i * 2)

# NHANH: gom vào list rồi chuyển một lần
kq = np.array([i * 2 for i in range(10000)])

# NHANH NHAT: vector hóa hoàn toàn
kq = np.arange(10000) * 2
```

- Dùng `float32` thay `float64` khi làm Deep Learning → tiết kiệm một nửa bộ nhớ

Bảng tra cứu nhanh

Việc cần làm	Lệnh
Tạo mảng từ list	<code>np.array([1,2,3])</code>
Mảng số 0 / số 1	<code>np.zeros((m,n))</code> , <code>np.ones((m,n))</code>
Dãy số	<code>np.arange(a,b,buoc)</code> , <code>np.linspace(a,b,n)</code>
Số ngẫu nhiên	<code>rng = np.random.default_rng(42)</code>
Xem hình dạng	<code>a.shape</code> , <code>a.ndim</code> , <code>a.dtype</code>
Đổi kiểu	<code>a.astype(np.float32)</code>
Lấy hàng / cột	<code>a[i]</code> , <code>a[:, j]</code>
Lọc theo điều kiện	<code>a[a > 5]</code>
if-else cho mảng	<code>np.where(cond, x, y)</code>
Sao chép thật sự	<code>a.copy()</code>
Thống kê	<code>a.sum()</code> , <code>a.mean()</code> , <code>a.std()</code>
Theo hàng / cột	<code>a.sum(axis=1)</code> , <code>a.sum(axis=0)</code>
Vị trí lớn nhất	<code>a.argmax()</code>
Đổi hình dạng	<code>a.reshape(m, -1)</code> , <code>a.T</code>
Ghép mảng	<code>np.vstack</code> , <code>np.hstack</code> , <code>np.concatenate</code>
Nhân ma trận	<code>A @ B</code>
Khoảng cách	<code>np.linalg.norm(a - b)</code>
Sắp xếp	<code>np.sort(a)</code> , <code>np.argsort(a)</code>

Notebook thực hành

File: `numpy_practice.ipynb`

Notebook đi kèm buổi học, gồm 10 phần bài tập từ dễ đến khó, có sẵn ô code để chạy thử và ô TODO để tự làm.

- **Cách 1 — Google Colab** (khuyến nghị, không cần cài gì):
 - Vào `colab.research.google.com` → Tập → Tải notebook lên
- **Cách 2 — máy cá nhân:**

```
pip install numpy jupyter
jupyter notebook numpy_practice.ipynb
```

- **Cách 3 — VS Code:** mở thẳng file `.ipynb` (cần extension Python + Jupyter)

Quan trọng

Hãy **tự gõ lại** từng dòng code thay vì copy-paste. Gõ tay giúp nhớ cú pháp nhanh hơn nhiều lần.

Bài tập tại lớp

- 1 Tạo mảng các số chẵn từ 0 đến 20, in ra tổng và trung bình cộng
- 2 Tạo ma trận 3×3 chứa các số từ 1 đến 9, rồi in ra: hàng thứ hai, cột cuối cùng, và đường chéo chính
- 3 Cho mảng điểm của 10 học sinh, đếm số học sinh đạt (điểm ≥ 5) và in ra danh sách điểm của những em này
- 4 Tạo ma trận điểm 5 học sinh $\times$ 3 môn. Tính điểm trung bình mỗi học sinh, mỗi môn, và tìm học sinh có điểm trung bình cao nhất
- 5 Chuẩn hóa một mảng số bất kỳ về khoảng $[0, 1]$ theo công thức

$$x' = \frac{x - \min}{\max - \min}$$

Bài tập về nhà

- 1 Viết hàm `sigmoid(x)` và `relu(x)`, áp dụng lên mảng `np.linspace(-5, 5, 11)`
- 2 Cho hai mảng `y_that` và `y_du_doan`, viết hàm tính MSE và MAE (không dùng vòng lặp)
- 3 Tạo ma trận ngẫu nhiên 100×5 , chuẩn hóa z-score theo từng cột, kiểm tra lại trung bình ≈ 0 và độ lệch chuẩn ≈ 1
- 4 Tạo "ảnh" ngẫu nhiên kích thước $(64, 64, 3)$ kiểu `uint8`. Chuyển sang ảnh xám, lật ngang, cắt vùng giữa 32×32
- 5 Cài đặt k-NN đơn giản: sinh 50 điểm 2 chiều với nhãn 0/1, dự đoán nhãn của một điểm mới bằng 3 láng giềng gần nhất
- 6 **Nâng cao:** viết lại bài hồi quy tuyến tính gradient descent, in giá trị loss sau mỗi 100 bước và nhận xét

Tóm tắt buổi học

- **ndarray** — mảng nhiều chiều, cùng kiểu, nằm liên tục trong bộ nhớ → nhanh
- **Vector hóa** — tính toán trên cả mảng, không cần vòng lặp
- **Broadcasting** — tự động khớp hình dạng giữa các mảng khác kích thước
- **shape và axis** — hai khái niệm phải nắm thật chắc; luôn `print(x.shape)`
- **View vs copy** — slicing dùng chung bộ nhớ; muốn độc lập thì `.copy()`
- **@** — nhân ma trận, phép toán nền tảng của mạng nơ-ron

Buổi sau

Pandas — xử lý dữ liệu dạng bảng, và **Matplotlib** — trực quan hóa dữ liệu.

Tài liệu: numpy.org/doc/stable/user/absolute_beginners.html